

PVT 掌锐电子

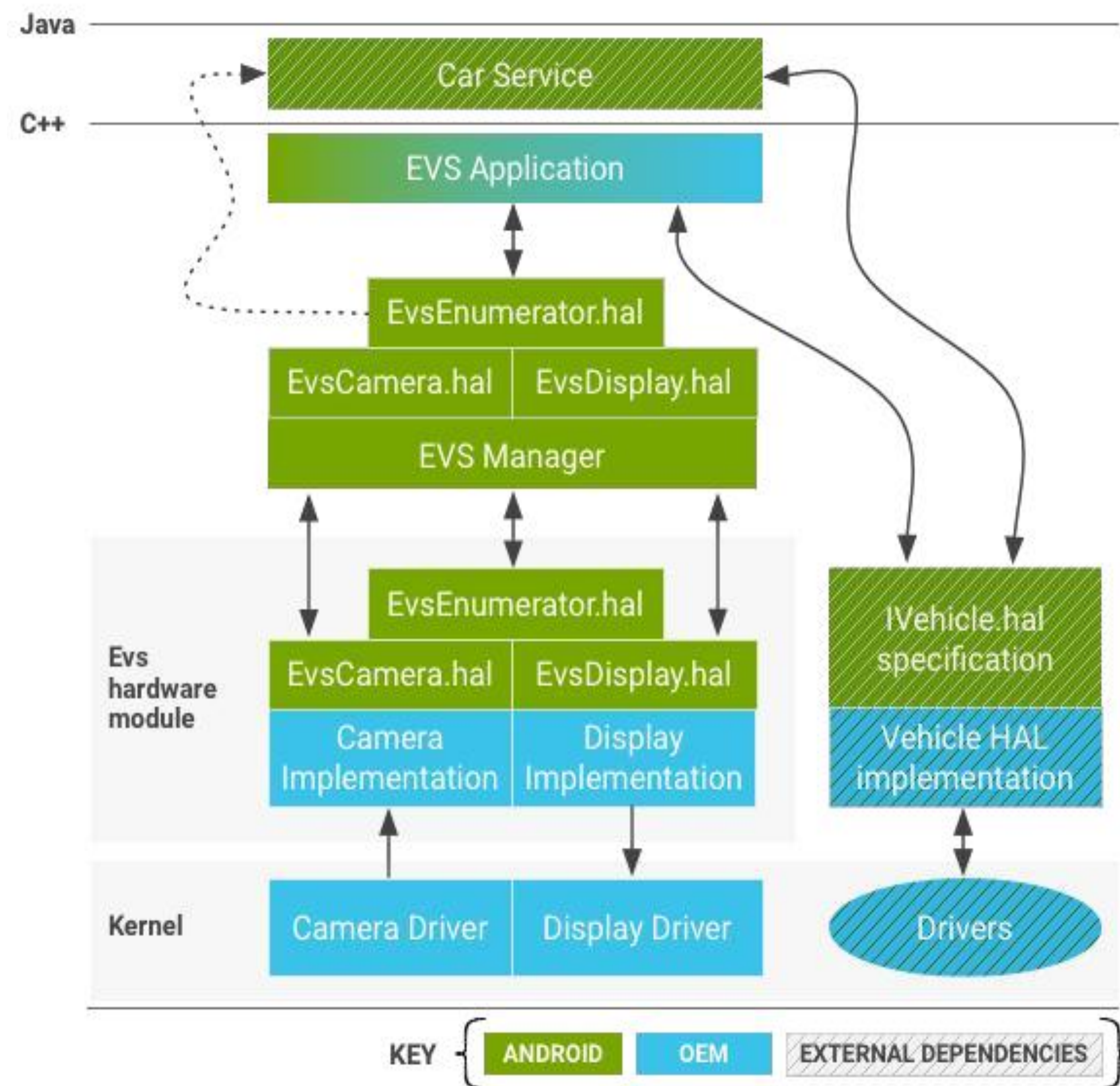
MT8676_EVS介绍

卢浩东

一, evs 介绍

EVS (Exterior View System) 外部视景系统: 是 Android 为车载开发的 camera 架构, 主要针对汽车外景系统, 如倒车影像、360 全景等。

参考谷歌文档介绍, EVS主要包括 **EVS APP**、**EVS Manager**、**EVS HAL**。如下架构图所示APP通过EVS Manager与EVS HAL通信, 获取操作camera与display的权限与代理对象,通过代理对象 (IEvsCamera和IEvsDisplay), 实现camera与display的交互, 将camera的图像及时的显示到display 上。



二、EVS代码路径与编译产物

8676单系统

EvsHal实现:

hardware/interfaces/automotive/evs/aidl/impl/default/

编译的产物: android.hardware.automotive.evs-aidl-default-service

push位置: vendor/bin/hw

EvsManager实现:

packages/services/Car/cpp/evs/manager/aidl/

编译的产物: evsmanagerd

EvsAppdemo示例:

packages/services/Car/cpp/evs/apps/default/

编译的产物: evs_app

push位置: system/bin/

8678_3OS

EvsHal实现:

vendor/mediatek/proprietary/hardware/external/evs/

编译的产物: evshal-service

push位置: vendor/bin/hw

EvsManager实现:

packages/services/Car/cpp/evs/manager/aidl/

编译的产物: evsmanagerd

EvsAppdemo示例:

packages/services/Car/cpp/evs/apps/default/

编译的产物: evs_app

push位置: system/bin/

三、Evshal介绍

Evshal总体框架：

1. 顶层服务接口

android.hardware.automotive.evs

这是 Android 定义的汽车电子视觉系统框架接口，提供了 EVS 服务的标准 API。基于 AIDL 的默认服务实现，用于进程间通信。

2. EVS 核心服务模块

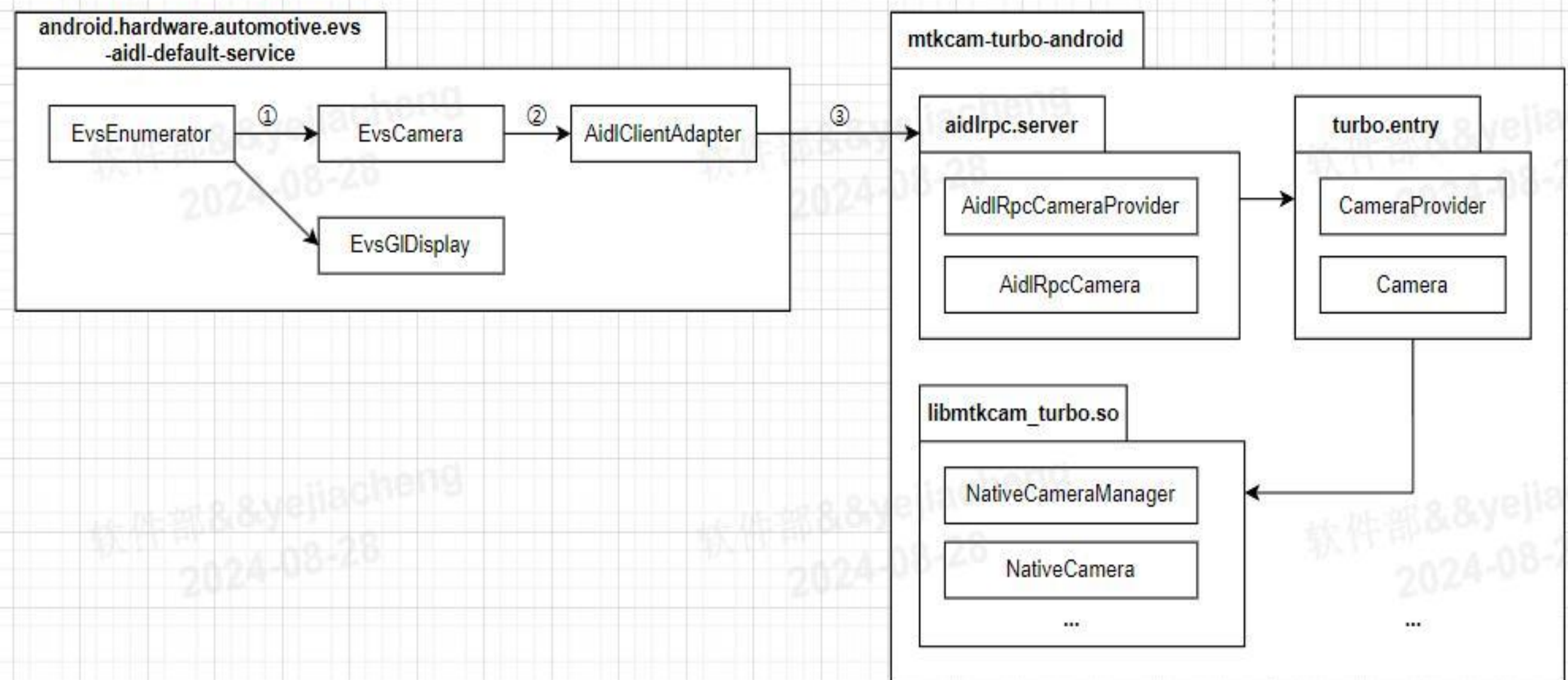
EvsEnumerator：负责枚举系统中可用的 EVS 设备

EvsCamera:提供相机控制接口，如开启、关闭、设置参数等。

AidClientAdapter: AIDL 客户端的适配器，用于将 EVS 服务调用转发到 IRPC 层。

3 MTK 平台相关层

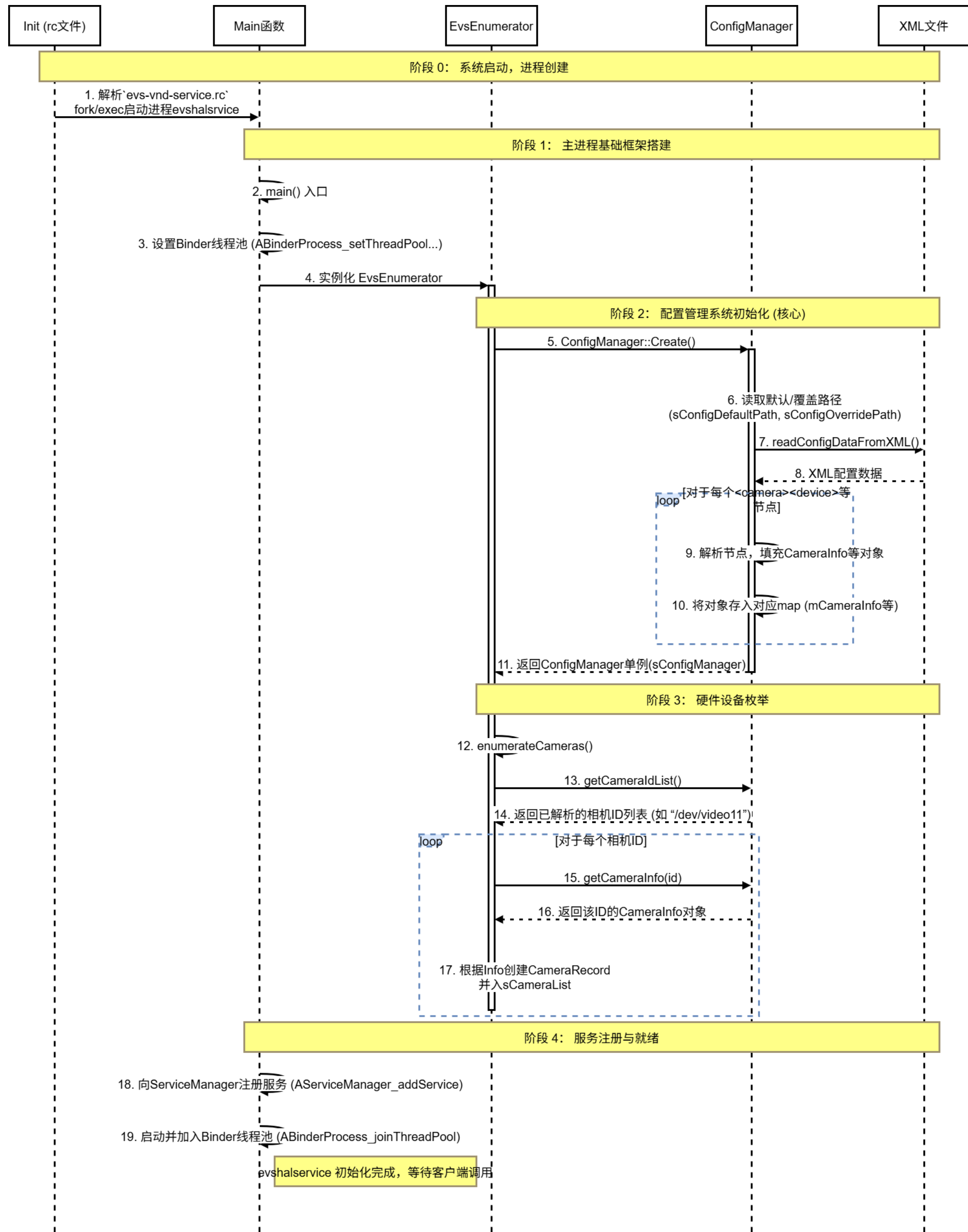
mtkcam-turbo-android:MediaTek平台的相机模块。



三、Evshal介绍

1:evshalservice初始化

配置解析与加载，读取XML文件，将静态配置转化为内存中的结构化数据，为后续所有操作提供“地图”，配置数据从XML文件流向 ConfigManager内部Map，再根据ID流向 EvsEnumerator的sCameraList，时序图如右图所示



三、Evshal介绍

2:Camera的打开与初始化

通过 EvsEnumerator 的 openCamera 得到 EvsCamera 实例，EvsEnumerator 主要函数如下

// 获取可用摄像头列表

1)ScopedAStatus getCameraList();

// 获取指定摄像头的流配置

2)ScopedAStatus getStreamList();

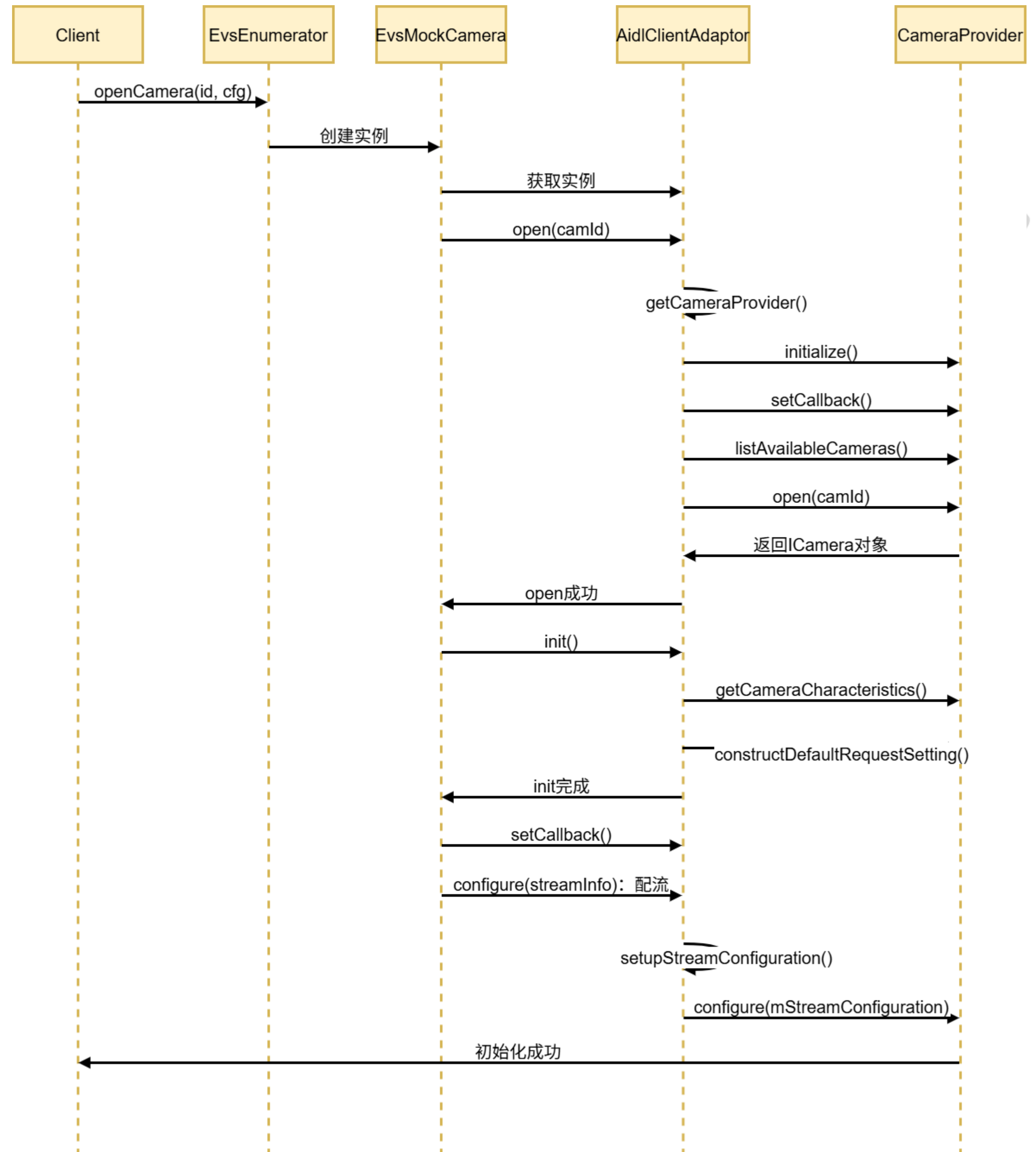
// 打开摄像头设备

3)ScopedAStatus openCamera();

// 关闭摄像头设备

4)ScopedAStatus closeCamera();

opencamera时序图如右：



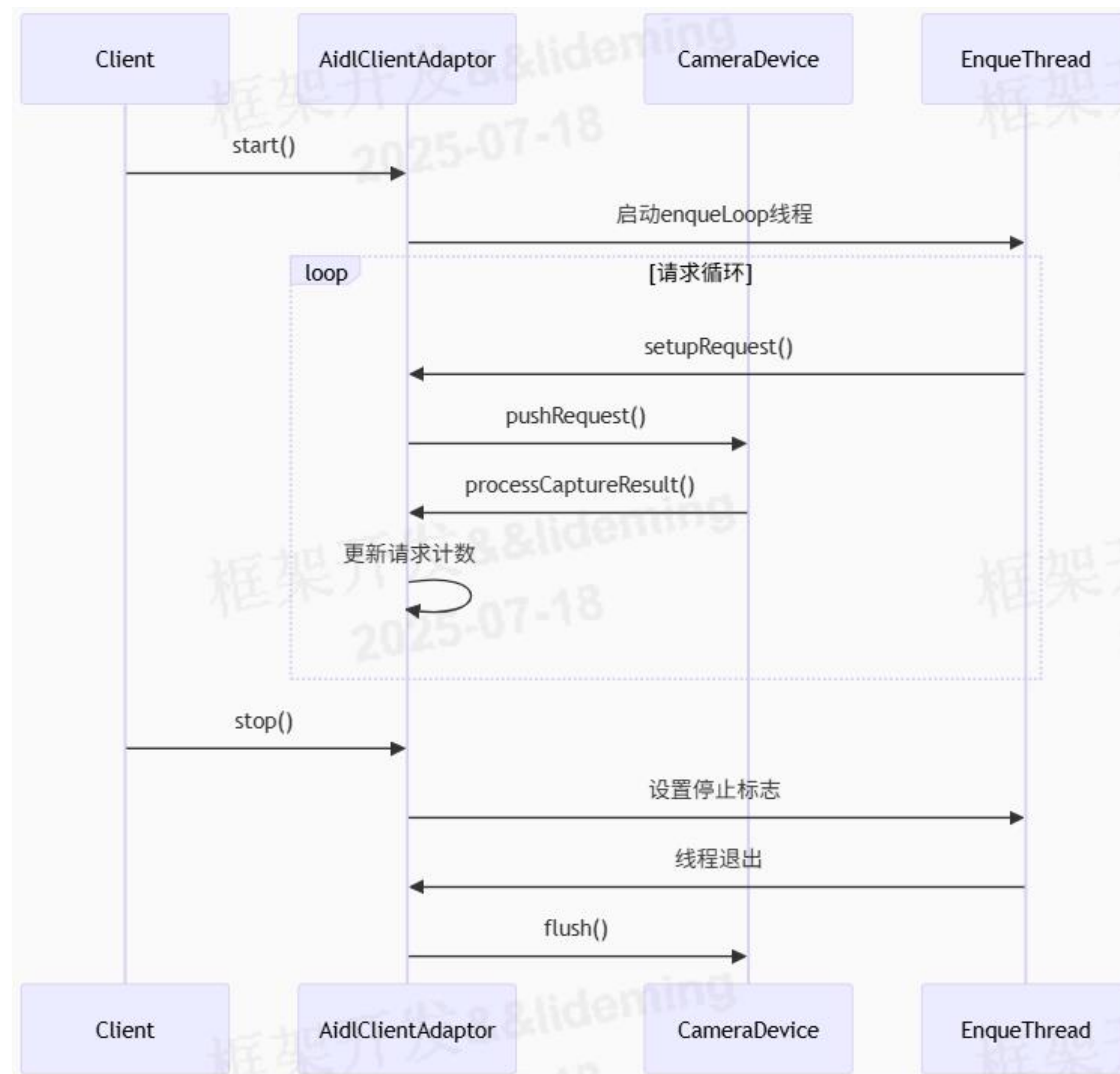
三、Evshal介绍

3: 视频流处理

openCamera后拿到EvsMockCamera对象，此对象又依赖于AidlClientAdapter。

AidlClientAdapter通过aidl接口与 mtkcam-turbo-android的aidlrpc server 通信，

AidlClientAdapter取流时会创建一个enqueueLoop线程，线程不断向AidlRpcCamera发送请求，图像数据会填充到请求buffer中并通过processCaptureResult 方法回调给AidlClientAdapter。



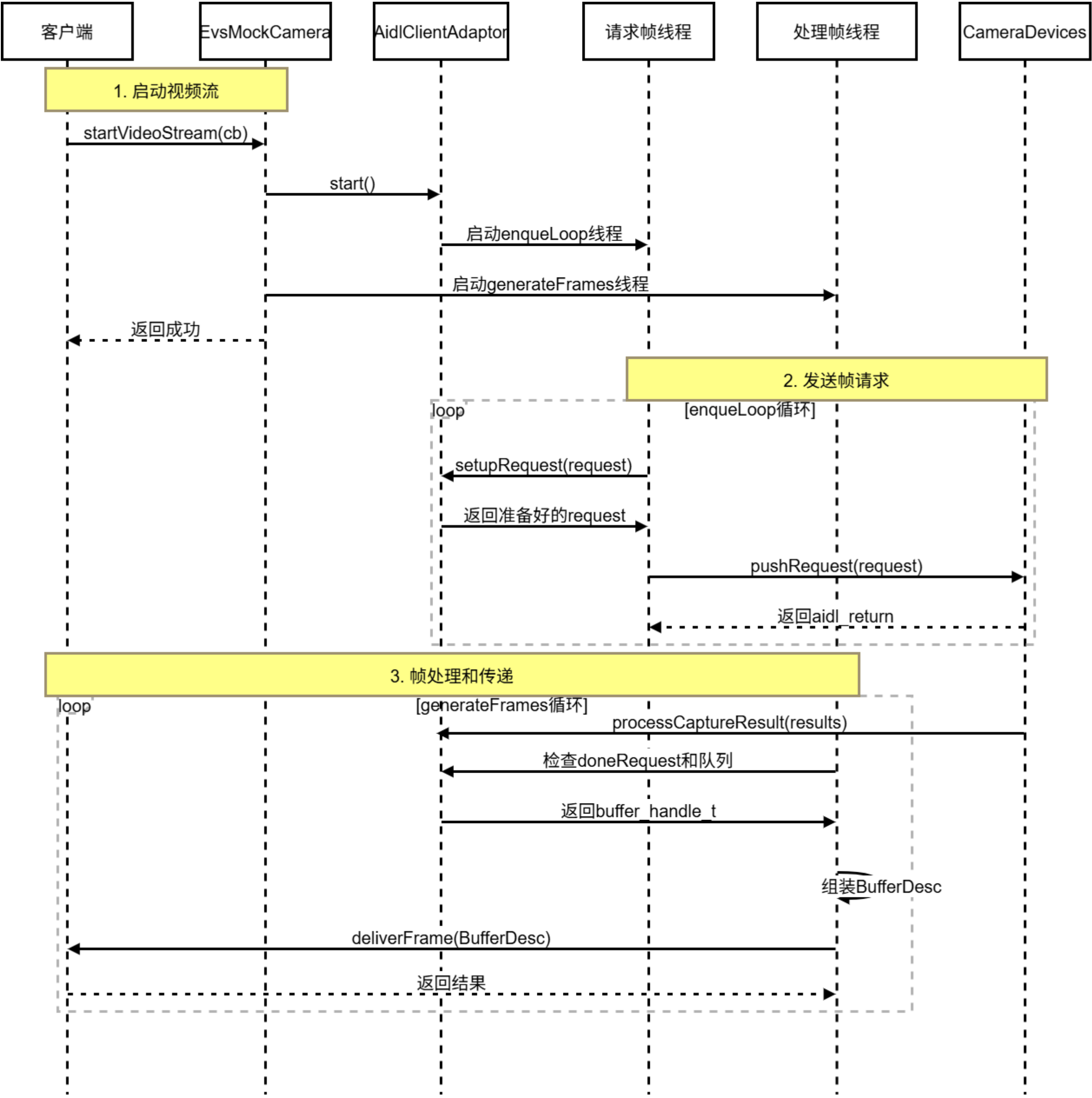
三、Evshal介绍

3. 1: 启动流

EVs相机视频流启动过程采用分层架构与生产者-消费者模式，实现了高效的图像数据流水线处理。系统通过双线程并行、异步回调、缓冲区池管理等技术手段

生产者线程 (enqueueLoop): 请求帧捕获，循环执行setupRequest()准备捕获请求调用mCamera->pushRequest()提交请求至硬件层管理缓冲区状态转换（NOUSE → USING）。

消费者线程 (generateFrames): 处理完成帧，传递至客户端，监听完成队列mBuffResultQue的帧到达事件从processCaptureResult()回调中获取完成的buffer_handle_t组装BufferDesc数据结构，包含图像元数据，通过deliverFrame()回调给客户端



三、Evshal介绍

3.2:关闭流

客户端调用 stopVideoStream()

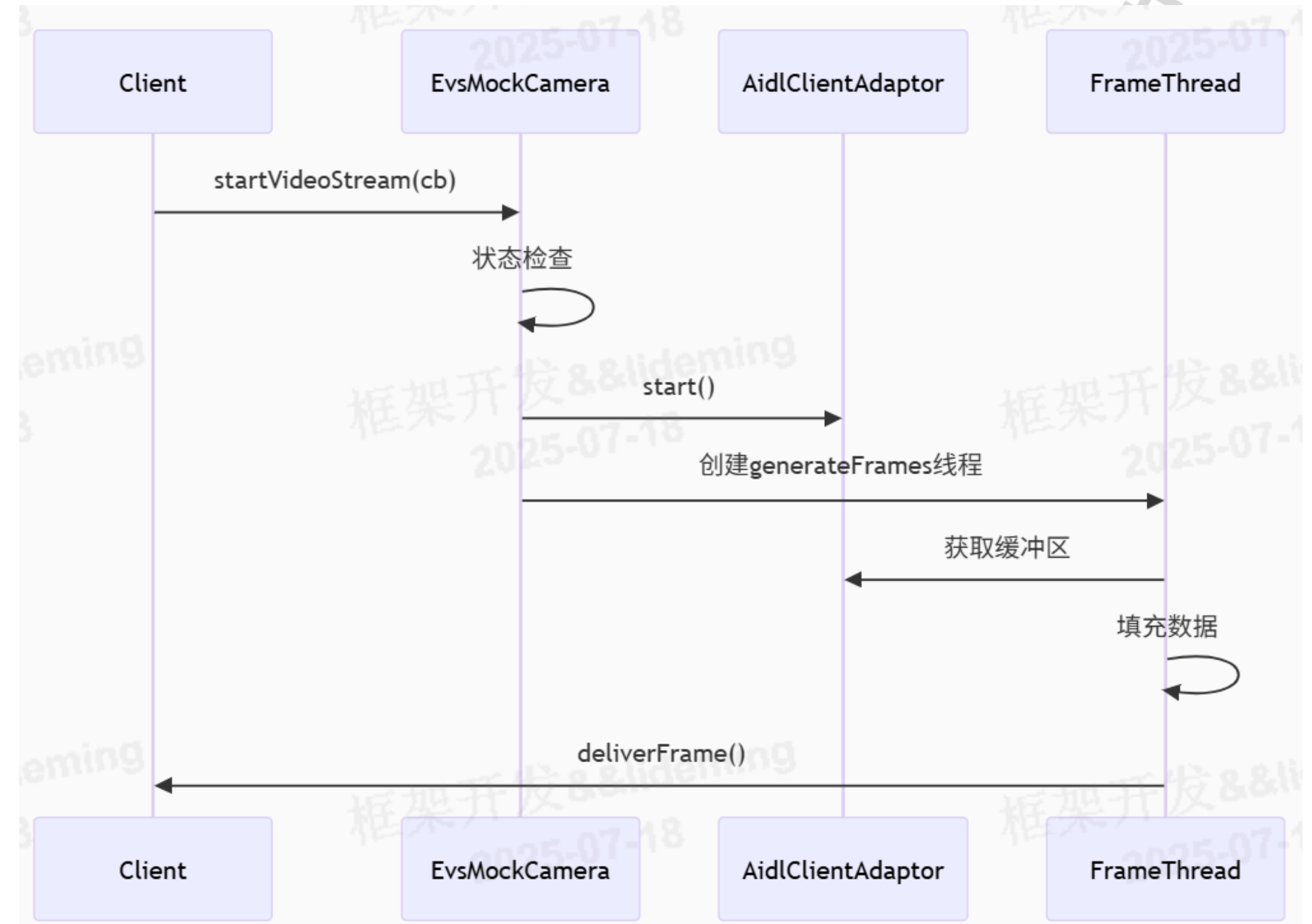
EvsMockCamera 接收请求

生产者线程检查到停止标志

消费者线程检查到 mStreamState !=
RUNNING, 退出循环

设置最终状态 mStreamState =
STOPPED, 清理资源

关流时序图如下, 就是停止线程, 修改
状态值, 释放buffer



四、问题案例

1、mtk 提供的 evshal 代码中 EvsCamera 只会去打开 camera id 0 的摄像头，取流配置为 1280x720 RGBA8888，在验证 evs 通路时需要修改下，比如验证 camera 1 取 1280x720 yuyv 数据修改如下：

```
EvsEnumerator.cpp  EvsMockCamera.cpp 7 X  AidlClientAdaptor.cpp 7  Android.bp .../in
interfaces > automotive > evs > aidl > impl > default > src > EvsMockCamera.cpp > {} aidl > {} android > {}
56 } // namespace
57
58 namespace aidl::android::hardware::automotive::evs::implementation {
59
60 EvsMockCamera::EvsMockCamera(
61     [[maybe_unused]] Sigil sigil,
62     const char* id,
63     std::unique_ptr<ConfigManager::CameraInfo>& camInfo)
64     : mFramesAllowed(0),
65       mFramesInUse(0),
66       mStreamState(STOPPED),
67       mCameraInfo(camInfo) {}
68 LOG(INFO) << "[tinyevs] show fun " << __FUNCTION__;
69
70 /* set a camera id */
71 mDescription.id = id;
72 auto camId = covertCameId(id);
73 camId = 1;
74 if (-1 == camId) {
75     LOG(ERROR) << "convertCameId failed, please check input id";
76     return;
77 }
78 /* set camera metadata */
79 if (camInfo) {
80     uint8_t* ptr = reinterpret_cast<uint8_t*>(camInfo->characteristics);
81     const size_t len = get_camera_metadata_size(camInfo->characteristics);
82     mDescription.metadata.insert(mDescription.metadata.end(), ptr, ptr + len);
83 }
84 mAidlClientAdapt = AidlClientAdaptor::getIntanceface();
85 CHECK_CONDITION(mAidlClientAdapt);
86 CHECK_CONDITION(mAidlClientAdapt->open(camId));
87 CHECK_CONDITION(mAidlClientAdapt->init());
88 mAidlClientAdapt->mCamera->setCallback(mAidlClientAdapt);
89 // Initialize parameters.
90 initializeParameters();
91 mAidlClientAdapt->configure();
92 LOG(INFO) << " [tinyevs] AidlClientAdaptor config done";
93
```

```
EvsEnumerator.cpp  EvsMockCamera.cpp 7  AidlClientAdaptor.cpp 7 X  Android.bp .../impl  Android.
interfaces > automotive > evs > aidl > impl > default > src > AidlClientAdaptor.cpp > setupStreamConfiguration()
100
101 bool bool_aidl_return = false;
102 setupStreamConfiguration();
103 mCamera->configure(mStreamConfiguration, &mHalConfiguration,
104                 &bool_aidl_return);
105 LOG(INFO) << "[tinyevs]AidlClientAdaptor configure -";
106 return true;
107 }
108 // get from evs_mock_configuration.xml
109 void AidlClientAdaptor::setupStreamConfiguration() {
110     mStreamConfiguration.sessionParams = mRequestTemplate;
111     Stream stream;
112     stream.id = 0 /*STREAM_HD*/;
113     stream.streamType = StreamType::OUTPUT /*OUTPUT*/;
114     stream.width = 1280;
115     stream.height = 720;
116     // stream.height = bufferProp.format differ format need convert
117     stream.format = EImageFormat::eImgFmt_YUY2;
118     stream.usage = GRALLOC_USAGE_HW_TEXTURE | GRALLOC_USAGE_HW_CAMERA_WRITE |
119                 GRALLOC_USAGE_SW_READ_RARELY | GRALLOC_USAGE_SW_WRITE_RARELY;
120     stream.dataSpace = Dataspace::V0_JFIF;
121     stream.transform = 0;
122     stream.bufPlanes.count = 0;
123     mStreamConfiguration.streams.emplace_back(stream);
124
125     mStreamConfiguration.operationMode = StreamConfigurationMode::NORMAL_MODE;
126     LOG(INFO) << "[tinyevs] set buffrt prop Width " << stream.width << "Height "
127                 << stream.height << "Format " << (int)stream.format;
128 }
129
```

PVT

Thank You

www.pvetec.com